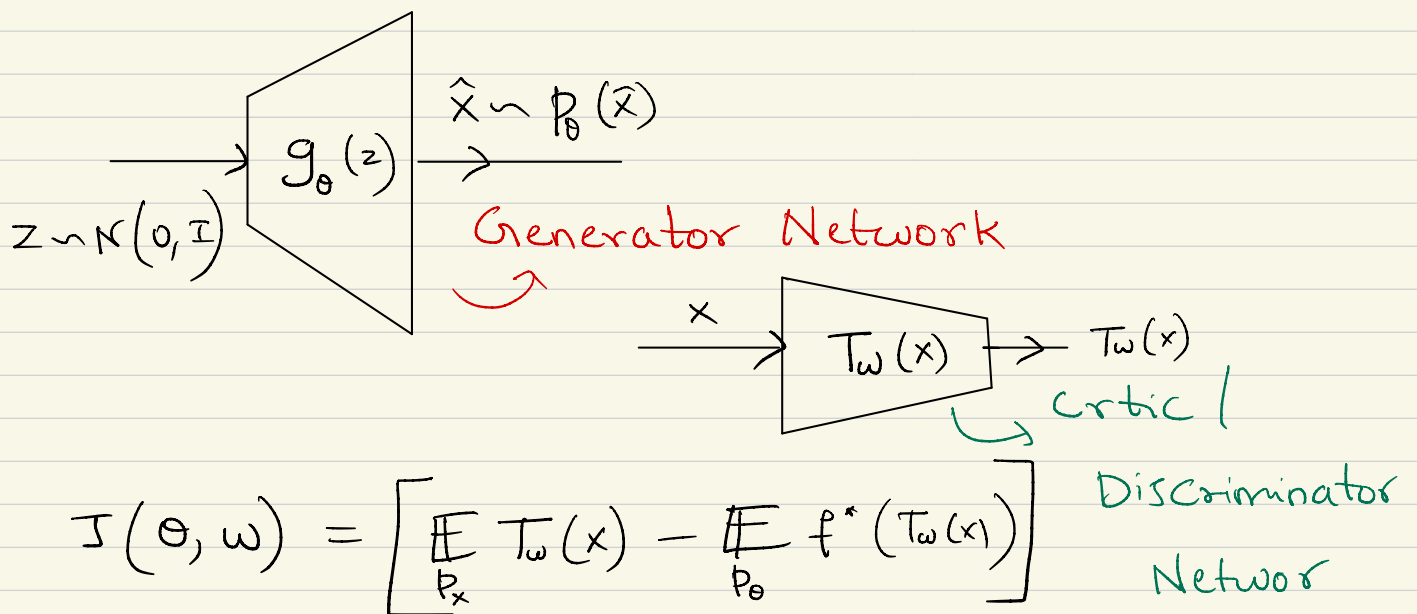


Implementing VDM for Generative Modeling



$$\theta^*, w^* = \underset{\theta}{\operatorname{argmin}} \max_w J(\theta, w)$$

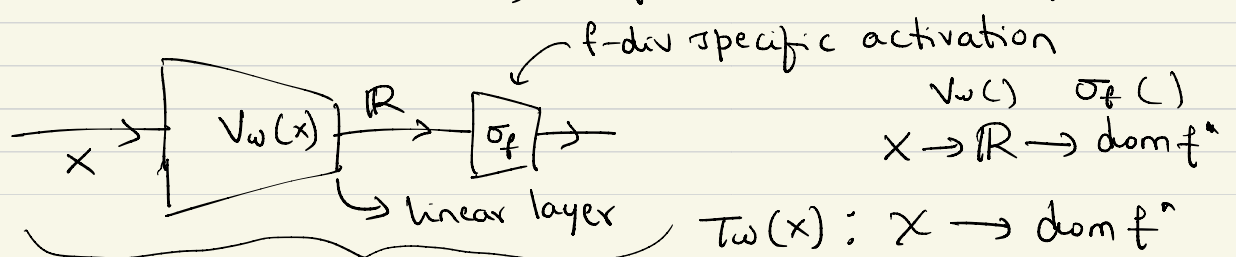
↳ Saddle point optimization.
Adversarial problem.

$$T_w(x) : \mathcal{X} \rightarrow \operatorname{dom} f^*$$

$$T_w(x) = \sigma_f(V_w(x)) \quad \text{where}$$

σ_f is a f -divergence specific activation

$$V_w(x) : \mathcal{X} \rightarrow \mathbb{R}, \quad \sigma_f(v) : \mathbb{R} \rightarrow \operatorname{dom} f$$



$$J(\theta, \omega) = \mathbb{E}_{p_x} \left[\sigma_f(v_\omega(x)) \right] - \mathbb{E}_{p_\theta} \left[f^*(\sigma_f(v_\omega(x))) \right]$$

Generative Adversarial Networks (GANs)

for GANs, the f -divergence is as follows:

$$f(u) = u \log u - (u+1) \log(u+1) \quad (\text{similar to JSD})$$

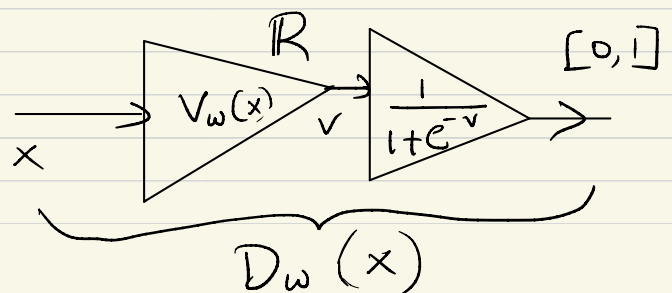
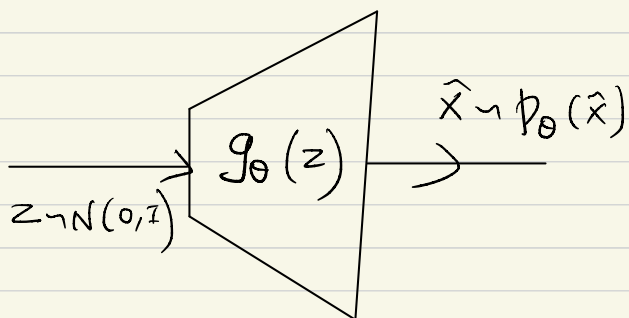
$$f^*(t) = -\log(1 - \exp(t)), \quad \text{dom } f^* = \mathbb{R}^-$$

$$\sigma_f(v) = -\log(1 + e^{-v})$$

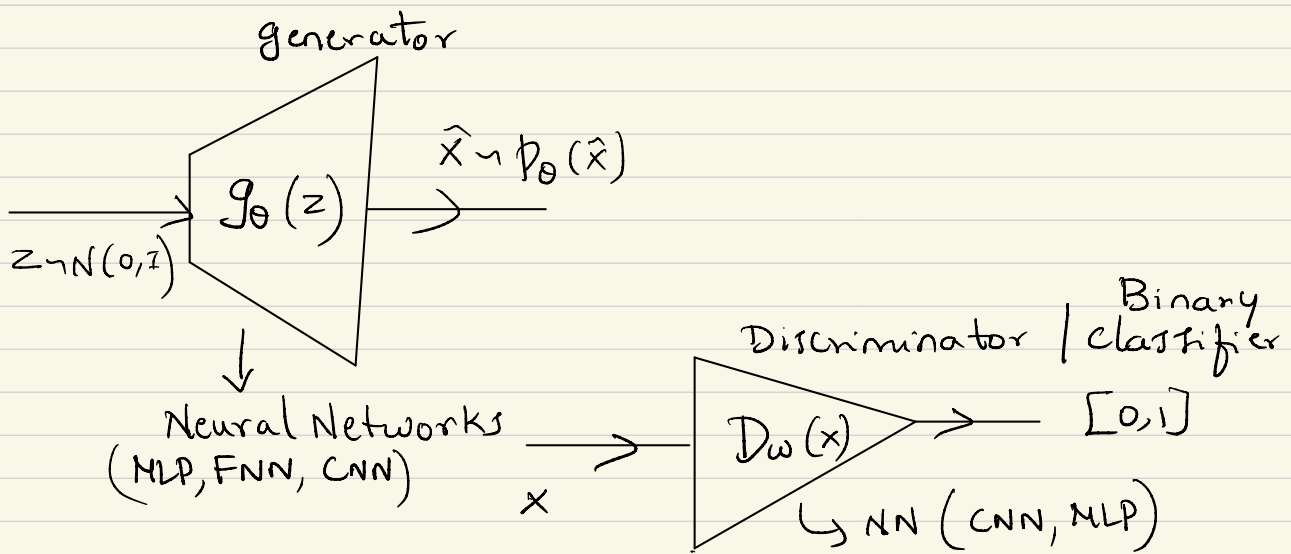
$$J(\theta, \omega) = \mathbb{E}_{p_x} \left[\sigma_f(v_\omega(x)) \right] - \mathbb{E}_{p_\theta} \left[f^*(\sigma_f(v_\omega(x))) \right]$$

$$J_{\text{GAN}}(\theta, \omega) = \mathbb{E}_{p_x} \left[\log D_\omega(x) \right] + \mathbb{E}_{p_\theta} \left[\log(1 - D_\omega(x)) \right]$$

where $D_\omega(x) = \frac{1}{(1 + e^{-v_\omega(x)})}$: sigmoid function



GAN Architecture



$$J_{\text{GAN}}(\theta, w) = \mathbb{E}_{x \sim p_x} \log D_w(x) + \mathbb{E}_{\hat{x} \sim p_\theta} \log(1 - D_w(\hat{x}))$$

Implementation of GAN in practice

Input: $D = \{x_1, x_2, x_3, \dots, x_n\} \sim \text{iid } p_x$

$$W^* = \arg \max_w \mathbb{E}_{p_x}(\log D_w(x)) + \mathbb{E}_{p_\theta}(\log 1 - D_w(\hat{x}))$$

$$\approx \arg \max_w \left[\frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(\hat{x}_j)) \right]$$

$$x_1, \dots, x_{B_1} \sim p_x$$

$$\hat{x}_1, \dots, \hat{x}_{B_2} \sim p_\theta, \quad \hat{x}_j = G_\theta(z_j)$$

$$W^{t+1} \leftarrow W^t + \alpha_1 \nabla_W J_{\text{GAN}}(\theta, W) : \text{one grad. descent step through Discriminator}$$

$\downarrow$
 step-size

$$\theta^* = \underset{\theta}{\operatorname{argmin}} J_{\text{GAN}}(\theta, W)$$

θ is kept a constant

$$\approx \underset{\theta}{\operatorname{argmin}} \left[\frac{1}{B_1} \sum_{i=1}^{B_1} \log D_W(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log (1 - D_W(\hat{x}_j)) \right]$$

Independent of θ

$$\approx \underset{\theta}{\operatorname{argmin}} \left[\frac{1}{B_2} \sum_{j=1}^{B_2} \log 1 - D_W(g_\theta(z_j)) \right]$$

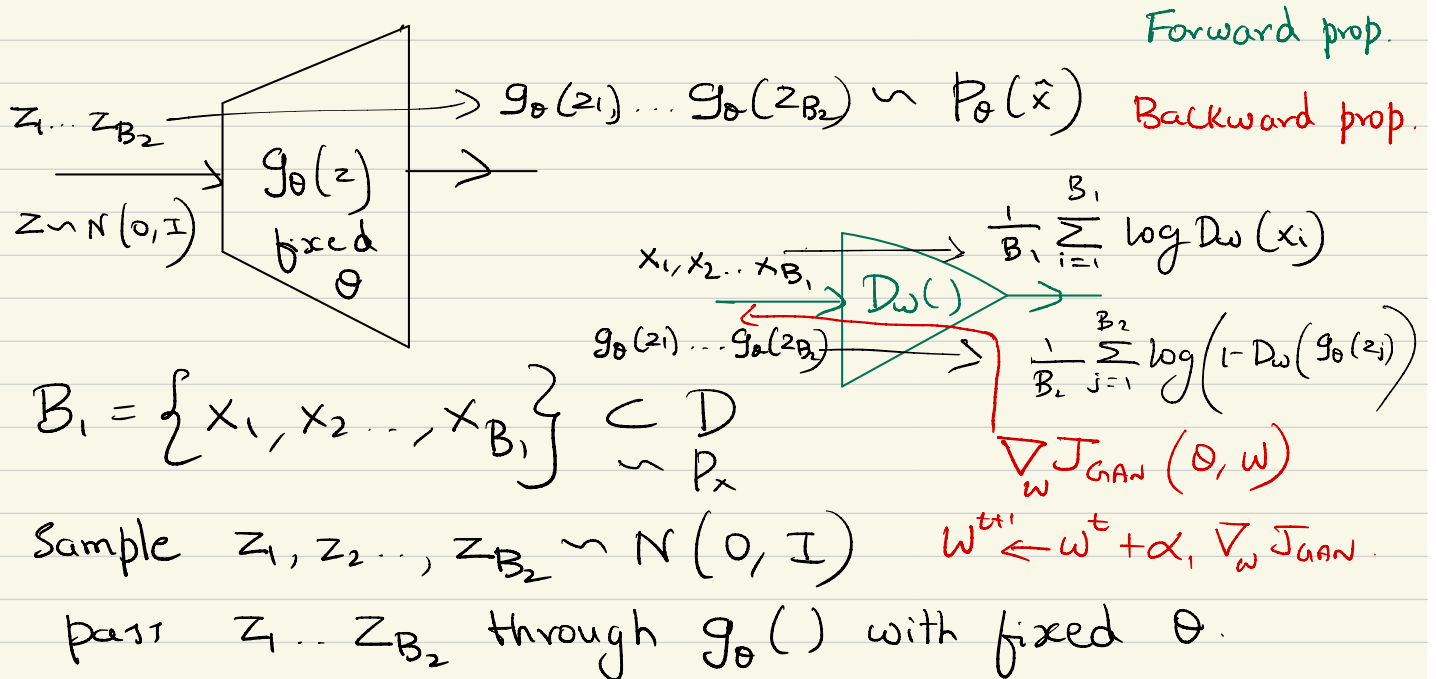
$$\theta^{t+1} \leftarrow \theta^t - \alpha_2 \nabla_\theta J_{\text{GAN}}(\theta, W) : \text{one grad. descent step through the generator}$$

$\hookrightarrow$ step size.

W is kept a constant.

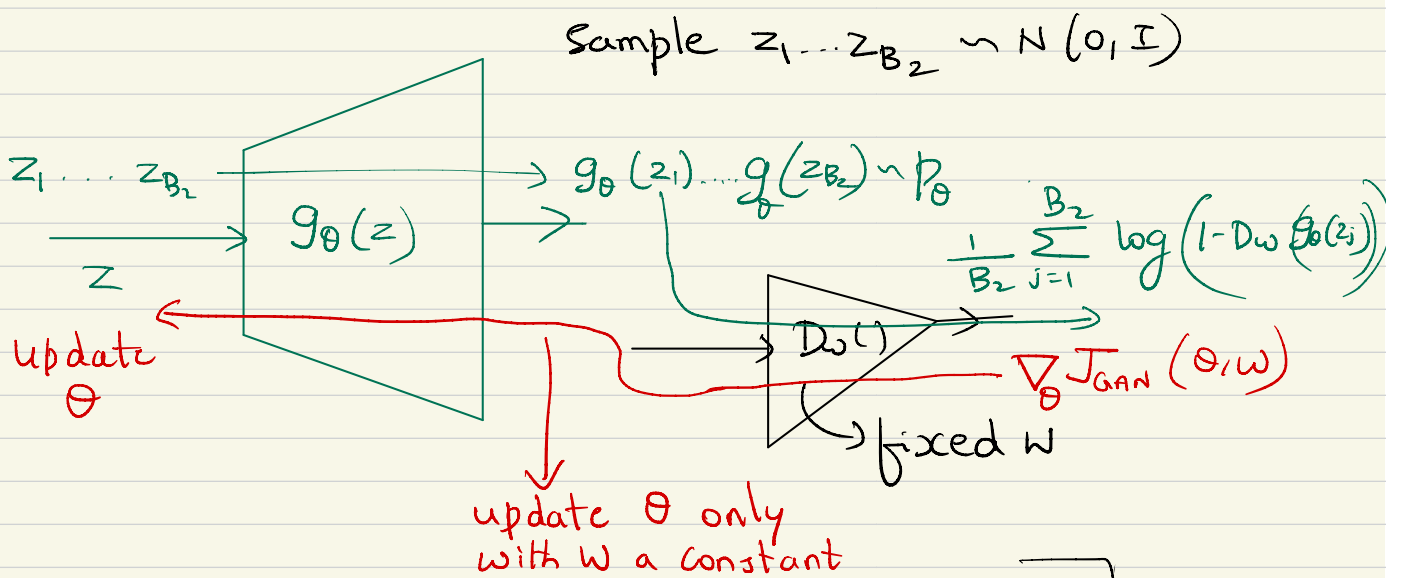
To train the Discriminator

keep θ constant, $D = \{x_1, x_2, \dots, x_n\}$



$$J_{GAN}(\theta, w) = \left[\frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(g_\theta(z_j))) \right]$$

To Train the Generator Network.



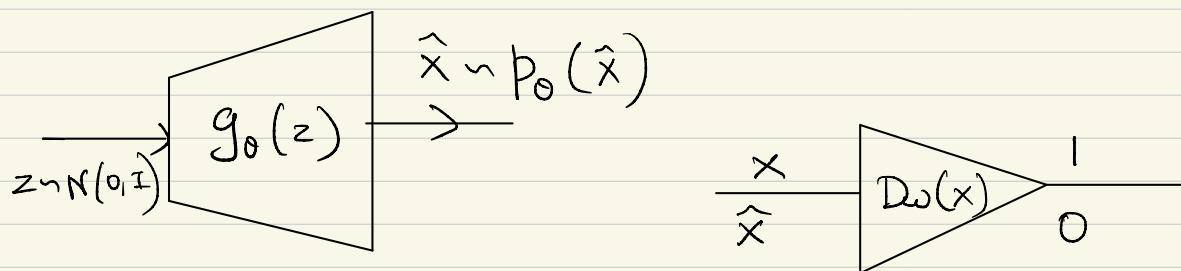
$$J_{\text{GAN}} = \left[\frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(g_\theta(z_j))) \right]$$

$$\theta^{t+1} \leftarrow \theta^t - \alpha_2 \nabla_\theta J_{\text{GAN}}(\theta, w)$$

Training VDM or GAN

Interpretation of a GAN as Classifier-Guided Generative Sampler

$$D = \{x_1, x_2, \dots, x_n\} \sim \text{iid } p_x$$



Goal : $p_0(\hat{x})$ to be close to $p_x(x)$.

Suppose there is a binary classifier

$$D_w(x) = \begin{cases} 1 & \text{if } x \sim p_x \\ 0 & \text{if } x \sim p_0 \end{cases}$$

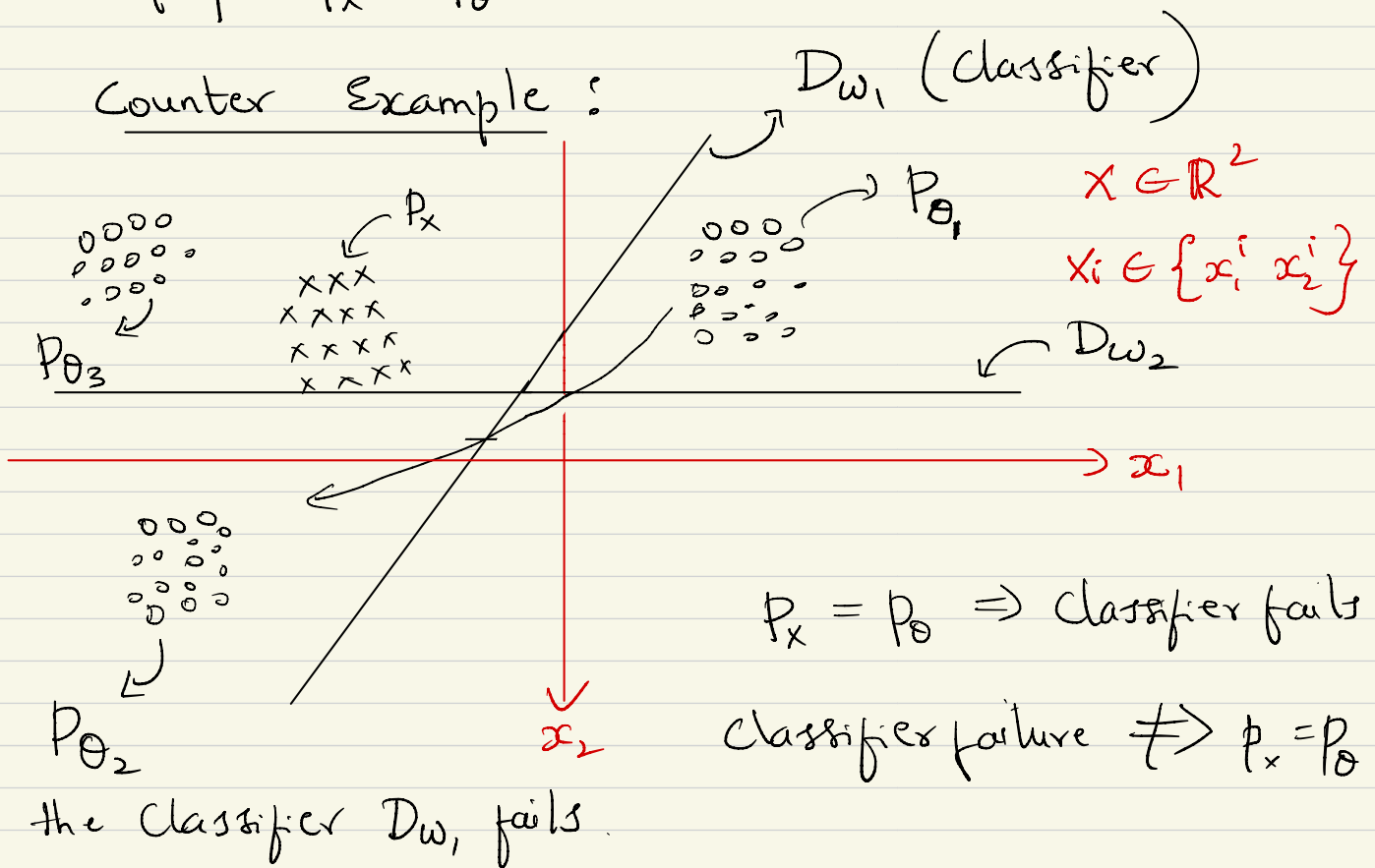
$D_w(x)$ is a binary classifier between the samples of p_x & p_0 .

question : Can $D_w(x)$ be used for making p_0 & p_x closer?

Answer : Tweak θ (parameters of g_0) till the classifier 'fails' to distinguish b/w the samples of p_x & p_0 .

However, failure of the classifier neednot imply $p_x = p_0$.

Counter Example :



Hence, the classifier has to be simultaneously tweaked along with the generator.

Formulation of classifier guided sampler

suppose $D_w : X \rightarrow [0, 1]$

Let $D_w(x)$ represent the likelihood of the sample x coming from p_x .

The objective is to maximize the log-likelihood of x coming from p_x .

$$w^* = \arg \max_w \left[\mathbb{E}_{x \sim p_x} \log D_w(x) \right] : \text{maximize the likelihood of } x \sim p_x.$$

Classifier should also maximize the likelihood of $\tilde{x}$ not coming from p_x when $\tilde{x} \sim p_0$.

$$\mathbb{E}_{\tilde{x} \sim p_0} \log(1 - D_w(\tilde{x})) : \text{likelihood that } \tilde{x} \not\sim p_x.$$

$$w^* = \arg \max_w \mathbb{E}_{\tilde{x} \sim p_0} \log[1 - D_w(\tilde{x})] : \text{maximizing the likelihood of } \tilde{x} \not\sim p_x$$

The combined objective for the classifier training:

$$w^* = \arg \max_w \left\{ \mathbb{E}_{x \sim p_x} (\log D_w(x)) + \mathbb{E}_{\tilde{x} \sim p_0} \log[1 - D_w(\tilde{x})] \right\}$$
$$J(\theta, w)$$

The objective for the generator $g_\theta(z)$ is such that the classifier has to "fail".

Invert the optimization for the classifier

$$\theta^* = \operatorname{argmin}_{\theta} J(\theta, w)$$

$$\theta^*, w^* = \operatorname{argmin}_{\theta} \operatorname{max}_w [J(\theta, w)]$$

adversarial optimization.

Deep-Convolution GAN (DCGAN)

Typical in a GAN,

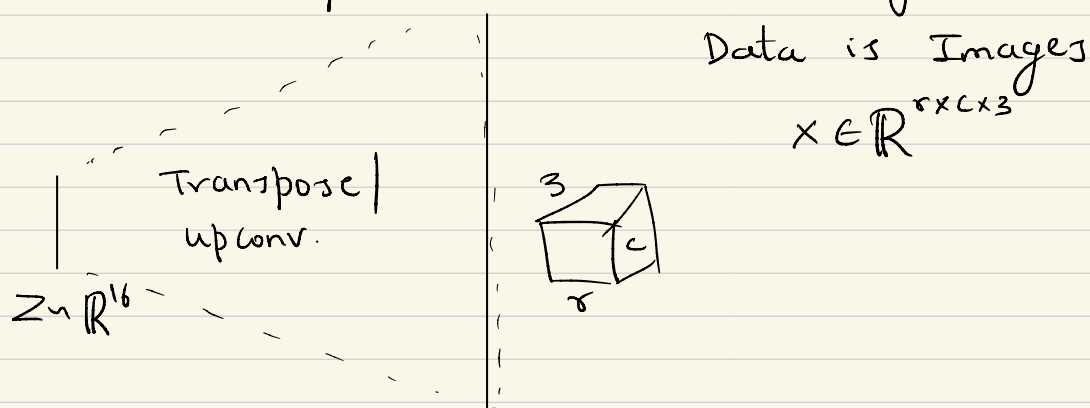
$$z \in \mathbb{R}^k, \quad x \in \mathbb{R}^d, \quad k \ll d.$$



Multi-layer perceptron (MLP)

$x \in \mathbb{R}^d \rightarrow$ reshape in the form of image

DC-GAN, upconvolutional or Transpose Convolutional layers are used in the generators.



Conditional GANs (C-GANs)

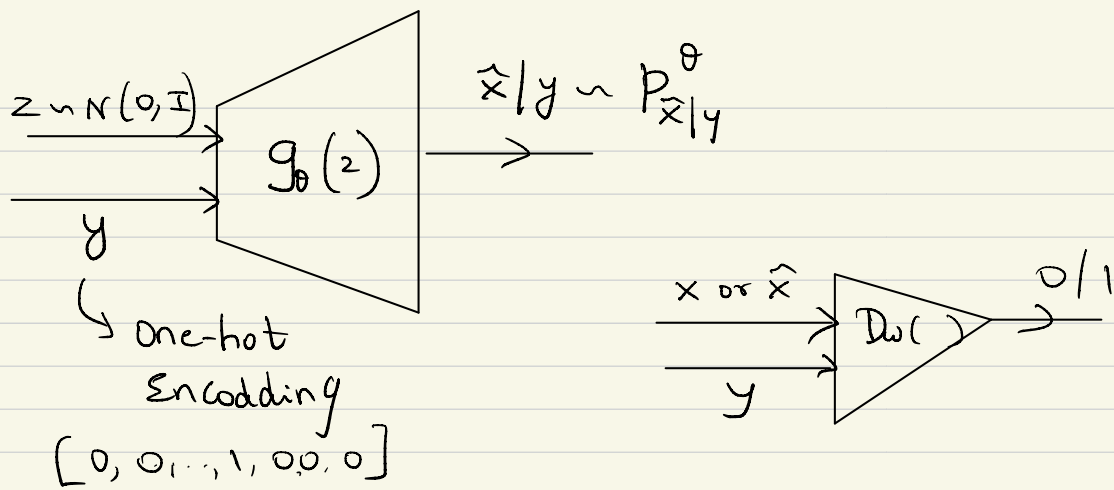
Data $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\} \sim \text{iid } p_{x,y}$

Eg: x : Images y : class-label (textual Embedding)
corrs. x corrs. x

Objective: Sample from the conditional distribution $p_{x|y}$ (instead of p_x)

Solution: Estimate $p_{x|y}$ & make p_θ approach $p_{x|y}$.

Modify the generator & discriminator to operate on the samples of the conditional distributions



$$J(\theta, w) = \mathbb{E}_{x, y \sim P_{x|y}} \log D_w(x, y) +$$

$$\mathbb{E}_{\hat{x}, y \sim P_{\hat{x}|y}^\theta} \log (1 - D_w(\hat{x}, y))$$

Inference with GANs/VDM

Input: A trained generator network g_{θ^*} .

Output: A sample from P_x which is not present in D .

